

“IT Will Never Approve That”

Constraint patterns — You Belong in the Loop — Sam Seim (they/them) — 2027 Nebraska Women in STEM Conference

START HERE

“No AI” is almost never the policy. It is the summary of a policy, relayed by someone who read the summary. The written version has edges — specific controls, specific classifications, specific vendors — and edges are things you can build to.

So the first move is never a workaround. It is a question, put to the person who owns the rule rather than the person who quoted it.

1 — FIND OUT WHAT THE RULE ACTUALLY SAYS

WHAT YOU WERE TOLD	WHAT IT USUALLY TURNS OUT TO BE	WHAT TO ASK — AND WHO TO ASK
“We don’t allow AI.”	An unapproved-vendor rule, or a data-classification rule. Rarely a ban on the category.	“Which control does this trip? Can you send me the clause?” — to whoever owns the policy, not whoever quoted it.
“Data can’t leave the network.”	A classification rule that covers some fields and not others. Derived and synthetic data are often out of scope, or simply unaddressed.	“Which classifications? Is derived or synthetic data in scope?” — data governance.
“Security won’t allow it.”	An unreviewed tool. Not a banned one — an unreviewed one. Nobody has done the paperwork.	“What does the review need from me? Can I see the questionnaire?” — infosec.
“It isn’t procured.”	True, and the most solvable of the five. Something on the approved list very often already does it.	“What’s already licensed that does this?” — procurement or vendor management.
“Legal said no.”	Frequently an unanswered question rather than a decided one — and the question they were asked was often not the one you would have asked.	“What was the question they were given?” — your manager, before you go anywhere near legal.

THE FOUR QUESTIONS TO HAVE READY

- 1 Which specific control?** Not “is this allowed” — which numbered rule am I tripping. A control number turns an argument into a task.
- 2 What classification is this data?** The answer is usually already written down and usually more granular than the person quoting it remembers.
- 3 What would approval require?** A questionnaire, a DPIA, a named owner, a retention answer. Ask for the list, then go get the list.
- 4 Who signs?** Find the actual signatory early. Everyone between you and them is relaying, and relays lose detail.

2 — DATA-HANDLING PATTERNS

- Move the reasoning, not the data.** Send the structure of the problem and keep the records where they are: schema, column names, data types, one synthetic row. What comes back is a transform you run inside the wall against the real data. The model never sees a record and never needs to.
- Build the synthetic exemplar deliberately.** Not a masked real row — masking leaks. Generate values from the schema: right type, right shape, right edge cases, wrong everything else. Keep it in version control, so what you send out is a reviewed artifact rather than an improvised one.
- Summarize inside, reason outside.** Where the model needs to see something about the data, compute the aggregate in the warehouse and send that. Counts, distributions, and outlier shapes are usually enough, and usually are not classified.

HOW TO ASK WITHOUT BECOMING A PROBLEM

- Ask in writing, ask for the clause, and thank them for it. You are not requesting an exception — you are trying to comply accurately, which is a thing security teams almost never get asked for and tend to remember fondly.
- Then design to what the clause actually says. A workflow built to a written control survives an audit, a reorganization, and the departure of the manager who approved it. A workaround survives none of those.

WHAT CAN LEAVE / WHAT CANNOT

- Check this against your own classification scheme — this is the common shape, not your rule.*
- Usually fine:** schema and field names, data types, row counts, business logic, error messages with values stripped, public documentation, your own code.
- Usually not:** any client or claimant record, anything with a natural key that maps back to a person, free-text notes (they contain everything), screenshots of production, and log excerpts nobody has read line by line.
- The one people miss:** prompts and outputs are records too. If the response contains regulated data, the response is regulated data, and it is now sitting in a chat history somebody has to account for.

ASK EVERY VENDOR THESE FOUR, IN WRITING

Is our input used for training? · How long is it retained, and where? · Which sub-processors touch it? · Can we get a zero-retention or enterprise tier, and what does it change? — The answers differ by tier, they change between contract renewals, and the free tier's answer is almost never the enterprise tier's answer. Get it in the contract, not the marketing page.

3 — NETWORK AND TOOLING

Inspected traffic is not blocked traffic. A proxy that reads your requests is a logging problem, not a prohibition. Find out whether it records payloads; if it does, that is a real constraint on what you send, and worth knowing before rather than after.

Use what is already licensed. The fastest approval is the one already granted. An enterprise agreement signed eighteen months ago frequently includes the capability you are about to spend a quarter requesting. Ask vendor management for the inventory.

Keep the model at the edge, not in the middle. Deterministic code moves the data; the model advises. That boundary is easy to draw on a whiteboard for a reviewer, easy to log, and it is what makes the next two conversations short.

Assume no egress and design for it. If the environment genuinely cannot reach the internet, the work does not stop — it moves. Build the artifact outside on synthetic data, review it, carry it in through whatever the sanctioned path is, and run it inside. This is slower and entirely normal.

Automate the boring eighty first. Most of what looks like an AI problem is a parsing problem wearing a hat. Do the deterministic majority with ordinary code, which needs no approval at all, and reserve the model for the fraction that genuinely needs judgment. Smaller ask, faster yes.

Log it like you'll be asked. Input, prompt version, model and version, timestamp, output, who approved. Build it as a first-class output, not a debug log. If you cannot reconstruct a decision two years from now, you do not have a system — you have a habit.

4 — PROCUREMENT, AND THE REVIEW PACKET

Ride an existing contract. A new capability under a signed agreement is an amendment. A new vendor is a procurement cycle. These are not the same amount of work and the difference is usually two quarters.

Scope the pilot to boring data. Propose the first build on data nobody is nervous about — public filings, your own documentation, synthetic sets. It gets approved because there is nothing to refuse, and it produces the working example that makes the second conversation about evidence instead of speculation.

Name an owner in the proposal. Every reviewer's real question is who is accountable when this is wrong. Answer it before they ask it, in writing, with a person's name. Yours, probably.

HAVE THIS READY BEFORE YOU ASK

A one-page description of the workflow, in plain language, with the data flow drawn.

The data classification of every input and every output.

What the failure mode is, who it affects, and how it gets caught.

The verification step: who checks the output, how long it takes, and where that time comes from.

The retention and training answers from the vendor, in writing.

What happens when you leave. Nobody asks this. Answer it anyway.

5 — RED LINES

SOMETIMES THE ANSWER IS ACTUALLY NO. TAKE THE NO.

Everything above is about finding out what the rule really says. None of it is about getting around a rule that says what it means. There is a version of this sheet that is a guide to quiet non-compliance, and that version gets somebody fired — probably not the person who wrote it.

Stop if: the data is regulated and the control is explicit; the material is under litigation hold or legal privilege; you cannot produce an audit trail for an output that will be relied on; the output affects someone's money, health, housing, employment, or grade and there is no named human who checks it; or you find yourself designing to avoid detection rather than to meet a rule.

The last one is the tell. If the design would embarrass you in a review, it is not a constraint pattern. It is a risk you have decided to carry personally, and you will be carrying it alone.